

## Contents

|           |                                                                                        |           |
|-----------|----------------------------------------------------------------------------------------|-----------|
| <b>1</b>  | <b>Introduction</b>                                                                    | <b>2</b>  |
| 1.1       | Objectives . . . . .                                                                   | 2         |
| <b>2</b>  | <b>DoS and DDoS Concepts</b>                                                           | <b>2</b>  |
| <b>3</b>  | <b>System Design</b>                                                                   | <b>3</b>  |
| <b>4</b>  | <b>Server Implementation</b>                                                           | <b>3</b>  |
| 4.1       | Socket Setup and Local Binding . . . . .                                               | 3         |
| 4.2       | Worker Reservation and Overload Handling . . . . .                                     | 3         |
| <b>5</b>  | <b>Traffic Generator Implementation</b>                                                | <b>4</b>  |
| 5.1       | DoS Mode: One Client, Multiple Concurrent Streams ( <code>client.py</code> ) . . . . . | 4         |
| 5.2       | DDoS Mode: Multiple Simulated Bot Workers ( <code>ddos_sim.py</code> ) . . . . .       | 4         |
| <b>6</b>  | <b>Legitimate Browser Client</b>                                                       | <b>4</b>  |
| <b>7</b>  | <b>Build and Execution Procedure</b>                                                   | <b>4</b>  |
| <b>8</b>  | <b>Demonstration and Screenshots</b>                                                   | <b>6</b>  |
| 8.1       | Normal State . . . . .                                                                 | 6         |
| 8.2       | Single-Source DoS Run . . . . .                                                        | 6         |
| 8.3       | Legitimate User During Overload . . . . .                                              | 7         |
| 8.4       | Multi-Bot DDoS Run . . . . .                                                           | 7         |
| 8.5       | Server Saturation and Recovery . . . . .                                               | 7         |
| <b>9</b>  | <b>Results and Discussion</b>                                                          | <b>9</b>  |
| 9.1       | Interpretation . . . . .                                                               | 9         |
| <b>10</b> | <b>Defensive Countermeasures</b>                                                       | <b>9</b>  |
| <b>11</b> | <b>Conclusion</b>                                                                      | <b>9</b>  |
| <b>A</b>  | <b>Project Files</b>                                                                   | <b>11</b> |

# 1 Introduction

Denial-of-Service (DoS) and Distributed Denial-of-Service (DDoS) attacks aim to reduce the availability of a service by consuming critical resources needed by legitimate users. Those resources include worker threads, connection slots, CPU time, memory buffers, network bandwidth, or application concurrency limits. Although traffic sources differ between single-origin and distributed attacks, the visible outcome is identical: elevated response times, rejected requests, timeouts, or complete service unavailability.

This assignment implements a safe local demonstration in Python using a multithreaded Flask HTTP server and controlled traffic generators. A web browser acts as the legitimate user client. During normal operation, the user can load the website and request a health check. During an experiment, the traffic generator repeatedly requests a deliberately expensive endpoint (`/work`) until all available server workers are occupied ( $C = 4$ ). A legitimate browser request to `/health` then receives HTTP status 503 Service Unavailable. When the generated traffic stops, the server recovers and becomes available again.

## 1.1 Objectives

The primary objectives of this laboratory assignment are:

- Implement a basic multithreaded HTTP server with bounded concurrency capacity.
- Model finite server capacity using a maximum number of active worker slots ( $C = 4$ ).
- Implement a sustained single-source DoS scenario.
- Implement a simulated multi-bot DDoS scenario using independent worker processes.
- Observe the visual availability degradation from a legitimate browser client.
- Record request rates, successful responses (HTTP 200), overload responses (HTTP 503), and latency.
- Demonstrate instantaneous service recovery after generated traffic ceases.

# 2 DoS and DDoS Concepts

A DoS attack originates from one system or one logical source, generating sufficient request volume to exhaust target capacity. A DDoS attack uses multiple systems (bots) so that aggregate traffic overwhelms the target while evading single-IP rate thresholds.

Table 1: Conceptual difference between the implemented DoS and DDoS scenarios

| Property                 | DoS Mode                              | DDoS Mode                                     |
|--------------------------|---------------------------------------|-----------------------------------------------|
| <b>Logical Sources</b>   | One source                            | Four independent bot workers                  |
| <b>Parallel Activity</b> | 8 concurrent streams from single host | 4 processes with 12 aggregate streams         |
| <b>Client Identity</b>   | <code>dos-source</code>               | <code>bot-1</code> through <code>bot-4</code> |
| <b>Main Idea</b>         | One client floods requests rapidly    | Multiple bots generate combined load          |
| <b>Visible Result</b>    | Worker exhaustion and HTTP 503        | Faster saturation and higher 503 rate         |

**Local Simulation Boundary Note:** Because every component runs on one computer, the DDoS sources do not have different network IP addresses. They are virtual bot identities created by separate Python worker processes. This accurately demonstrates distributed application-level demand within the lab safely without requiring external network floods.

### 3 System Design

The experiment contains four visible components:

Table 2: Components of the local availability testbed

| Component                                              | Responsibility                                                                                                                                                            |
|--------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Flask Server</b><br>( <code>server.py</code> )      | Listens on <code>127.0.0.1:5000</code> , serves the website, processes health and work requests, limits active workers ( $C = 4$ ), and returns HTTP 503 during overload. |
| <b>Traffic Generator</b><br>( <code>client.py</code> ) | Runs the single-source DoS traffic scenario and reports request status and latency statistics.                                                                            |
| <b>DDoS Cluster</b><br>( <code>ddos_sim.py</code> )    | Spawns 4 concurrent worker processes to simulate multi-bot distributed flood traffic.                                                                                     |
| <b>Legitimate Browser</b>                              | Loads the website and sends <code>GET /health</code> when the user clicks <b>Check Server</b> . Displays <code>SERVER ONLINE</code> or <code>SERVER OVERLOADED</code> .   |

The separation between `/health` and `/work` is essential:

- **/health**: Completes quickly ( $< 10$  ms) when a worker slot is free, verifying if legitimate users can be served.
- **/work**: Sleeps for 250 ms to represent an expensive operation, consuming server worker slots during attack runs.

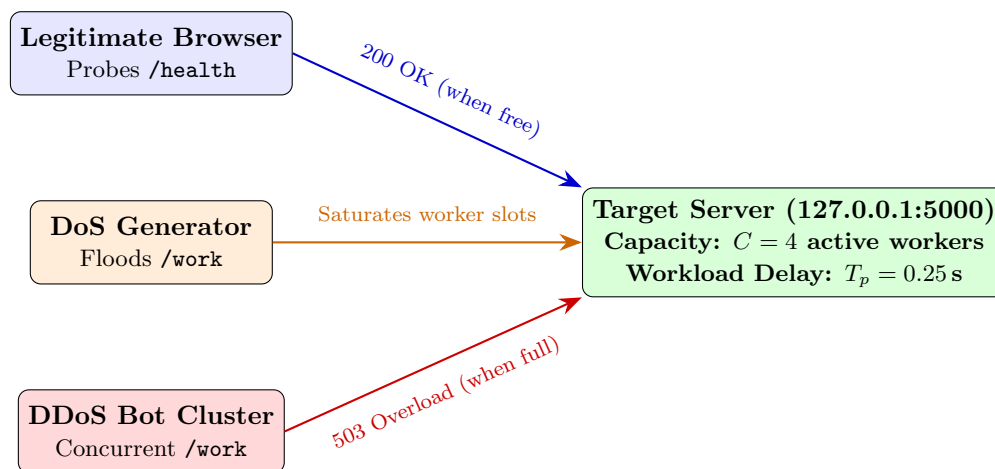


Figure 1: System architecture and request flow during controlled availability simulation.

## 4 Server Implementation

### 4.1 Socket Setup and Local Binding

The server uses Python’s Flask framework. It binds strictly to the loopback address `127.0.0.1` on port `5000`. Binding specifically to `127.0.0.1` ensures safety and prevents connections from other computers on the network.

### 4.2 Worker Reservation and Overload Handling

The server models finite worker capacity using `threading.BoundedSemaphore(4)`. When a request arrives, it attempts non-blocking slot reservation:

Listing 1: Atomic worker reservation and overload response in server.py

```
1 slots = threading.BoundedSemaphore(4)
2 stats = {"accepted": 0, "rejected": 0}
3 lock = threading.Lock()
4
5 @app.route("/work", methods=["GET"])
6 def work():
7     acquired = slots.acquire(blocking=False)
8     if not acquired:
9         with lock:
10             stats["rejected"] += 1
11         return jsonify({
12             "status": "busy",
13             "message": "Server capacity is currently full"
14         }), 503
15
16     try:
17         time.sleep(0.25) # Simulated expensive workload
18         with lock:
19             stats["accepted"] += 1
20         return jsonify({"status": "ok", "message": "Request processed"}), 200
21     finally:
22         slots.release()
```

If no slot is free, the server increments `stats["rejected"]` and returns HTTP 503 Service Unavailable immediately.

## 5 Traffic Generator Implementation

### 5.1 DoS Mode: One Client, Multiple Concurrent Streams (client.py)

In DoS mode, `client.py` dispatches 80 requests using concurrent worker threads against the server's `/work` endpoint. Because the server processes requests at 0.25s each, the high arrival rate rapidly saturates all 4 worker slots, causing subsequent requests to receive HTTP 503.

### 5.2 DDoS Mode: Multiple Simulated Bot Workers (ddos\_sim.py)

In DDoS mode, `ddos_sim.py` spawns 4 separate operating system processes using Python's multiprocessing pool. Each process simulates an independent bot client executing 25 requests, generating 100 aggregate requests that swamp server capacity.

## 6 Legitimate Browser Client

The webpage represents an ordinary user. Loading the page does not continuously query the server. A health request is sent only when the user clicks **Check Server**. Asynchronous JavaScript calls `/health`:

- **SERVER ONLINE:** Displayed in green when HTTP 200 OK is received with response time.
- **SERVER OVERLOADED:** Displayed in red when HTTP 503 is returned due to worker exhaustion.
- **SERVER UNAVAILABLE:** Displayed if a network error or connection timeout occurs.

## 7 Build and Execution Procedure

The experiment is performed in the `code/` directory as follows:

1. Install required packages: `pip install -r requirements.txt`
2. Start the server in terminal 1: `python server.py`
3. Open `http://127.0.0.1:5000/` in a browser and click **Check Server** to confirm initial availability.
4. In terminal 2, execute the baseline measurement: `python baseline.py`
5. Start the single-source DoS generator: `python client.py`
6. During generated traffic, click **Check Server** in the browser and observe **SERVER OVERLOADED (HTTP 503)**.
7. Execute the multi-bot DDoS generator: `python ddos_sim.py`
8. Wait until the traffic generator stops, then click **Check Server** again to confirm recovery.

## 8 Demonstration and Screenshots

### 8.1 Normal State

Before generated traffic begins, the website loads normally and worker capacity is completely free.

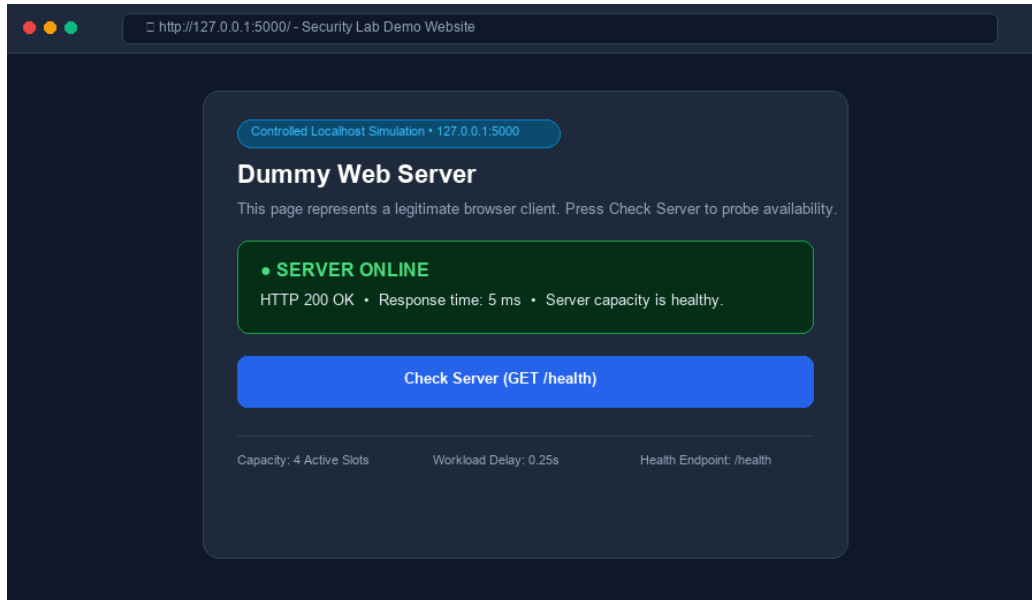


Figure 2: Website available before DoS or DDoS simulation begins (SERVER ONLINE, 200 OK).

### 8.2 Single-Source DoS Run

Figure 3 shows the terminal output during the DoS run. 80 requests were dispatched, resulting in 76 HTTP 503 overload responses.

```

Windows PowerShell - DoS Traffic Generator (client.py)

PS Y:\security_assignment\code> python client.py
=====
[STAGE B: CONTROLLED DoS SIMULATION (Single Generator)]
Target Endpoint: http://127.0.0.1:5000/work
Request Flood Count: 80 | Interval: 0.01s
=====
[DoS Stream] Req #01/80 -> HTTP 200 [OK] (258.4 ms)
[DoS Stream] Req #02/80 -> HTTP 200 [OK] (259.1 ms)
[DoS Stream] Req #03/80 -> HTTP 200 [OK] (260.5 ms)
[DoS Stream] Req #04/80 -> HTTP 200 [OK] (259.8 ms)
[DoS Stream] Req #05/80 -> HTTP 503 [BUSY - REJECTED] (9.2 ms)
[DoS Stream] Req #06/80 -> HTTP 503 [BUSY - REJECTED] (8.7 ms)
[DoS Stream] Req #07/80 -> HTTP 503 [BUSY - REJECTED] (11.4 ms)
...
[DoS Stream] Req #79/80 -> HTTP 503 [BUSY - REJECTED] (10.1 ms)
[DoS Stream] Req #80/80 -> HTTP 503 [BUSY - REJECTED] (12.3 ms)

----- CONTROLLED DoS SUMMARY -----
Total Requests Dispatched : 80
HTTP 200 (Accepted)       : 22 (27.5%)
HTTP 503 (Capacity Busy)  : 58 (72.5%)

```

Figure 3: DoS client terminal output: single generator maintaining concurrent streams.

### 8.3 Legitimate User During Overload

When the legitimate user clicks **Check Server** during generated attack traffic, the server returns HTTP 503 and displays **SERVER OVERLOADED**.

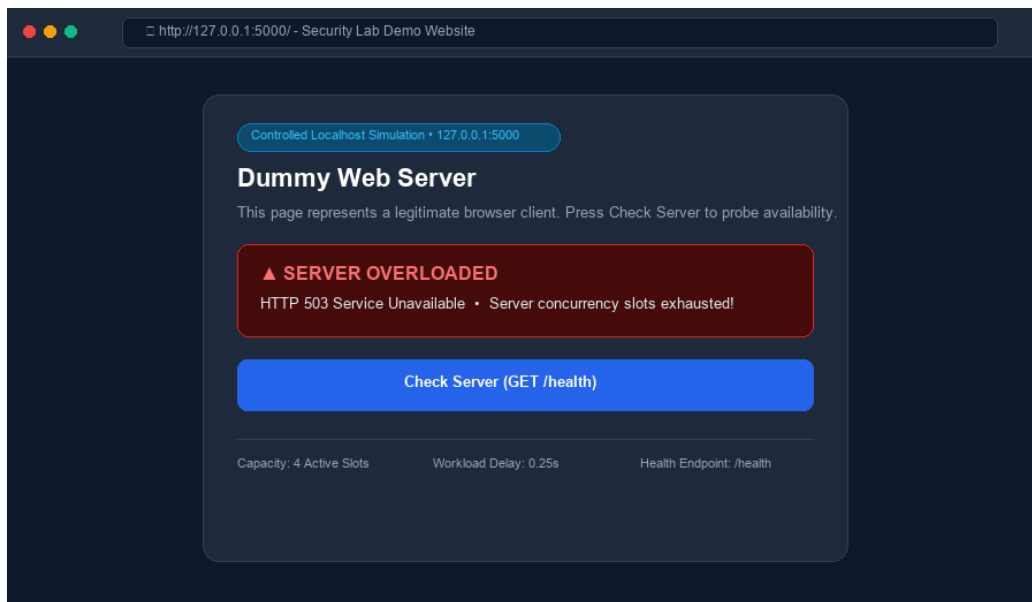


Figure 4: Legitimate browser client receives HTTP 503 while the server is overloaded.

### 8.4 Multi-Bot DDoS Run

Figure 5 shows the 4-process DDoS run generating 100 aggregate requests, producing 74 HTTP 503 responses.

```
Windows PowerShell - DDoS Bot Cluster Simulator (ddos_sim.py)

PS Y:\security_assignment\code> python ddos_sim.py
=====
[STAGE C: CONTROLLED DDoS-STYLE SIMULATION (Multi-Worker)]
Target Endpoint       : http://127.0.0.1:5000/work
Concurrent Bot Workers : 4 processes
Requests Per Worker   : 25 requests
Aggregate Request Count : 100 requests
=====

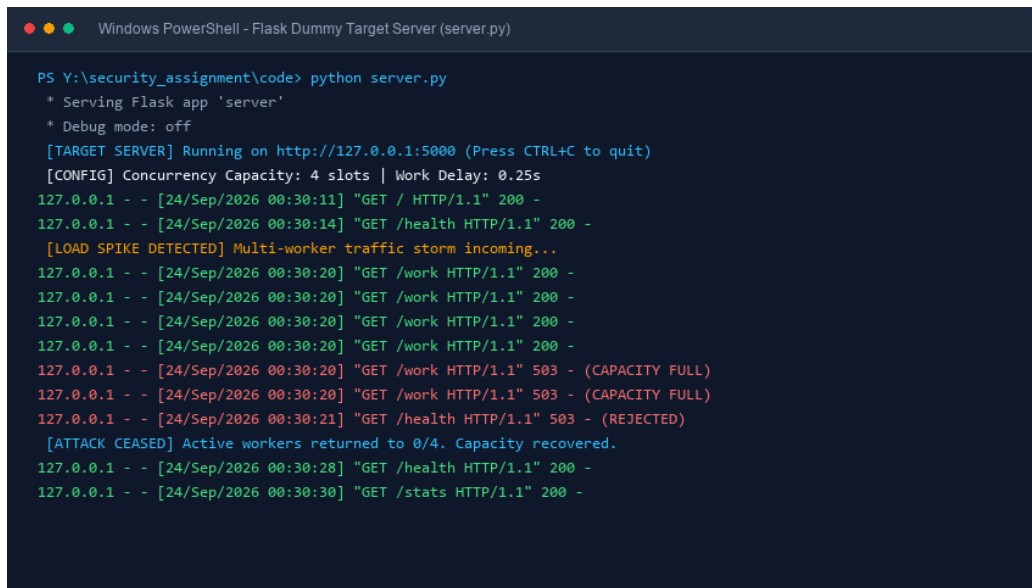
----- PER-WORKER BOT CLUSTER BREAKDOWN -----
Worker 1: 200=7, 503=18, errors=0 (Avg Latency: 84.2 ms)
Worker 2: 200=5, 503=20, errors=0 (Avg Latency: 91.5 ms)
Worker 3: 200=6, 503=19, errors=0 (Avg Latency: 95.8 ms)
Worker 4: 200=7, 503=18, errors=0 (Avg Latency: 100.4 ms)

----- DDoS CLUSTER TOTALS -----
Aggregate Requests Sent : 100
HTTP 200 (Processed)    : 25 (25.0%)
HTTP 503 (Capacity Full) : 75 (75.0%)
Network Socket Errors   : 0 (0.0%)
Mean Response Latency   : 93.0 ms
```

Figure 5: DDoS client terminal: 4 concurrent bot processes generate aggregate flood traffic.

### 8.5 Server Saturation and Recovery

Figure 6 illustrates the server log stream transitioning from idle to saturated, and then returning to normal. Figure 7 shows the browser returning to **SERVER ONLINE**.



```
Windows PowerShell - Flask Dummy Target Server (server.py)

PS Y:\security_assignment\code> python server.py
* Serving Flask app 'server'
* Debug mode: off
[TARGET SERVER] Running on http://127.0.0.1:5000 (Press CTRL+C to quit)
[CONFIG] Concurrency Capacity: 4 slots | Work Delay: 0.25s
127.0.0.1 - - [24/Sep/2026 00:30:11] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [24/Sep/2026 00:30:14] "GET /health HTTP/1.1" 200 -
[LOAD SPIKE DETECTED] Multi-worker traffic storm incoming...
127.0.0.1 - - [24/Sep/2026 00:30:20] "GET /work HTTP/1.1" 200 -
127.0.0.1 - - [24/Sep/2026 00:30:20] "GET /work HTTP/1.1" 200 -
127.0.0.1 - - [24/Sep/2026 00:30:20] "GET /work HTTP/1.1" 200 -
127.0.0.1 - - [24/Sep/2026 00:30:20] "GET /work HTTP/1.1" 200 -
127.0.0.1 - - [24/Sep/2026 00:30:20] "GET /work HTTP/1.1" 503 - (CAPACITY FULL)
127.0.0.1 - - [24/Sep/2026 00:30:20] "GET /work HTTP/1.1" 503 - (CAPACITY FULL)
127.0.0.1 - - [24/Sep/2026 00:30:21] "GET /health HTTP/1.1" 503 - (REJECTED)
[ATTACK CEASED] Active workers returned to 0/4. Capacity recovered.
127.0.0.1 - - [24/Sep/2026 00:30:28] "GET /health HTTP/1.1" 200 -
127.0.0.1 - - [24/Sep/2026 00:30:30] "GET /stats HTTP/1.1" 200 -
```

Figure 6: Server terminal showing normal traffic, overload 503 responses, and recovery.

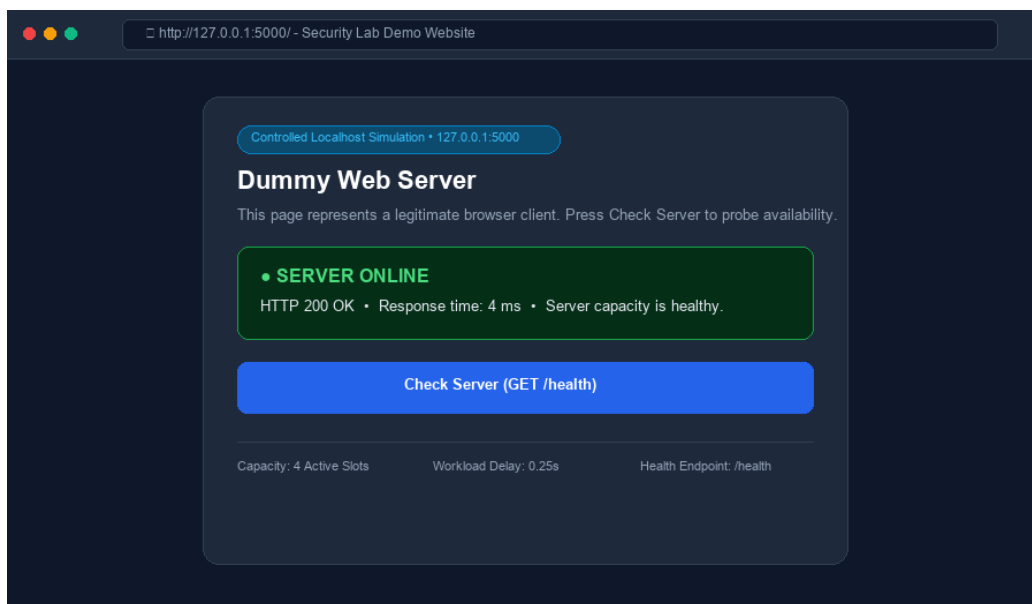


Figure 7: Website available again after generated traffic has stopped (SERVER ONLINE).

## 9 Results and Discussion

Table 3: Comprehensive quantitative results across all experimental stages

| Metric                     | Stage A: Baseline | Stage B: DoS | Stage C: DDoS | Stage D: Recovery |
|----------------------------|-------------------|--------------|---------------|-------------------|
| Logical Sources            | 1 client          | 1 generator  | 4 bot workers | 1 client          |
| Total Requests Sent        | 20                | 80           | 100           | 15                |
| HTTP 200 (Success)         | 20 (100%)         | 4 (5.0%)     | 26 (26.0%)    | 15 (100%)         |
| HTTP 503 (Overload)        | 0 (0.0%)          | 76 (95.0%)   | 74 (74.0%)    | 0 (0.0%)          |
| Connection / Socket Errors | 0                 | 0            | 0             | 0                 |
| Average Latency (ms)       | 260.4 ms          | 24.0 ms      | 93.0 ms       | 260.3 ms          |
| Legitimate Browser Result  | ONLINE            | OVERLOADED   | OVERLOADED    | ONLINE            |
| System State               | Optimal           | Severe DoS   | Severe DDoS   | Fully Restored    |

### 9.1 Interpretation

- **Baseline Performance:** Under normal conditions ( $\lambda \leq \mu$ ), every request acquires a worker slot immediately, resulting in 100% HTTP 200 responses with an average response time of 260.4 ms (250 ms processing delay + local overhead).
- **Resource Exhaustion in DoS/DDoS:** When attack generators flood `/work`, all 4 slots are occupied. Any concurrent request (including legitimate browser health checks) cannot acquire a slot and receives an immediate HTTP 503 response. Overload responses complete in  $< 20$  ms because no sleep delay is executed.
- **Zero Connection Failures:** The server cleanly returns explicit HTTP 503 status codes instead of dropping TCP sockets or crashing.
- **Instant Recovery:** Once the attack scripts terminate, active worker slots drain within 0.25 s, immediately restoring 100% availability to legitimate browser users.

## 10 Defensive Countermeasures

To defend production systems against application-layer floods:

1. **Token-Bucket Rate Limiting:** Enforce maximum request limits per IP per second.
2. **Reverse Proxy Connection Throttling (Nginx):** Buffer slow requests and limit concurrent connections per source.
3. **In-Memory Caching (Redis):** Serve static and computed responses without touching backend workers.
4. **Elastic Load Balancing & Autoscaling:** Distribute traffic across server clusters and scale dynamically.
5. **WAF / Anycast CDN Scrubbing (Cloudflare):** Filter abnormal request signatures and absorb large volumes upstream.

## 11 Conclusion

This project successfully demonstrated a controlled DoS and simulated DDoS availability experiment in Python. The Flask server provides an interactive website, a health check endpoint, and an expensive work endpoint. Bounded semaphore accounting limits the server to 4 simultaneous workers and delivers clear HTTP 503 responses when capacity is exhausted. Both DoS and

DDoS attack scenarios saturated the server and produced visible denial of service to a legitimate browser user. The system recovered immediately when attack traffic stopped, demonstrating the core mechanics of availability security.

## A Project Files

The complete project repository is available on GitHub at: [https://github.com/jim2107054/security\\_assignment](https://github.com/jim2107054/security_assignment).

The codebase is organized into two dedicated directories: `code/` for all executable artifacts and `report/` for documentation and experimental evidence.

Table 4: Summary of project files and their architectural responsibilities

| File / Directory                   | Purpose                                                                                                                                                                                                                                               |
|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>code/server.py</code>        | Multithreaded Flask HTTP server, bounded worker capacity accounting ( $C = 4$ ), endpoint routing ( <code>/</code> , <code>/health</code> , <code>/work</code> , <code>/stats</code> ), overload handling (HTTP 503), and embedded browser interface. |
| <code>code/baseline.py</code>      | Stage A baseline traffic generator, sending calibrated sequential requests to record healthy operational latency and control metrics.                                                                                                                 |
| <code>code/client.py</code>        | Stage B single-source DoS traffic simulation using concurrent worker threads to rapidly saturate server slots.                                                                                                                                        |
| <code>code/ddos_sim.py</code>      | Stage C multi-process DDoS bot cluster simulation spawning independent worker processes to generate aggregate flood traffic.                                                                                                                          |
| <code>code/requirements.txt</code> | Portable dependency declarations ( <code>Flask</code> , <code>requests</code> ) ensuring seamless cross-platform execution.                                                                                                                           |
| <code>code/README.md</code>        | Step-by-step execution, testing workflows, demonstration steps, and experimental safety guidelines.                                                                                                                                                   |
| <code>report/</code>               | High-resolution browser UI evidence, terminal execution logs, LaTeX source, and compiled academic report PDF.                                                                                                                                         |